

Orchestrating Multimodal Pipelines for Automated AI Video Generation

Reyyansh Bothra

May 2026

Abstract

End-to-end text-to-video diffusion models can produce visually impressive clips, but they offer limited control over narrative structure, shot composition, and multi-scene consistency. This paper reports on the design, implementation, and evaluation of an LLM-orchestrated pipeline that decomposes video generation into six discrete stages—script generation, shot planning, keyframe synthesis, video clip synthesis, narration generation, and automated assembly—coordinated by a large language model acting as a controller. We compare this pipeline against two simpler baselines (naive single-pass text-to-video generation and manual keyframe-plus-interpolation) across 30 short-form video briefs spanning product demos, narrative shorts, explainers, and social clips, using a five-point manual rubric covering prompt fidelity, temporal coherence, visual quality, and narration alignment. We find that (i) staged orchestration improves mean overall quality from 2.7 (naive) to 4.1 (orchestrated) at the cost of roughly $3\times$ higher end-to-end latency; (ii) the dominant failure modes are character/style drift across shots (40% of videos) and narration-duration mismatch (27%), both of which originate upstream of the video-synthesis stage rather than within it; and (iii) adding a CLIP-similarity validation-and-retry loop between stages cut prompt-misalignment failures from 47% to 20%. We discuss the resulting design implications, the limits of a small manually-graded study, and directions for reducing orchestration overhead.

1 Introduction

Generative video models have advanced rapidly, from early GAN-based frame synthesizers to large diffusion and transformer-based systems capable of producing multi-second, high-resolution clips directly from a text prompt [3, 4, 5, 6]. These end-to-end systems excel at single-shot visual fidelity but remain difficult to steer: a single prompt must encode subject, setting, camera motion, pacing, and narrative intent all at once, and there is little mechanism for enforcing consistency

across multiple shots or aligning visuals with a separately generated narration track.

A natural response, already common in production tooling and increasingly in research, is to treat video generation as a *pipeline* rather than a single model call: use a large language model (LLM) to plan the content (script, shots, timing), delegate each visual sub-task to a specialized generative model, and use an orchestration layer to sequence, validate, and retry these calls. This paper documents an exploration of that approach, built around four questions:

1. **Q1 (Architecture).** Does decomposing generation into LLM-planned stages improve prompt fidelity and shot-to-shot consistency relative to a single end-to-end generation call?
2. **Q2 (Failure modes).** Which failure modes dominate multi-shot AI-generated video, and at which pipeline stage do they originate?
3. **Q3 (Cost/latency).** What is the latency and compute cost of staged orchestration relative to single-pass generation, and where does time go?
4. **Q4 (Robustness).** How reliably can the pipeline run end-to-end without manual intervention, and does adding automated validation between stages change that reliability?

We do not claim to advance the state of the art in any individual generative component (script generation, image synthesis, or video synthesis); the contribution is a systems-level study of *orchestration*—how combining existing model classes through an LLM-driven controller changes the practical usability of AI video generation, and where the resulting pipeline still breaks down.

2 Background and Related Work

2.1 Text-to-video generation

Diffusion probabilistic models [1] underlie most recent text-to-video systems. Make-A-Video [3] and Imagen Video extended text-to-image diffusion to the

temporal domain by adding motion layers on top of pretrained image models. Stable Video Diffusion [4] popularized an image-to-video formulation, animating a single conditioning frame rather than synthesizing motion from text alone—a design choice directly relevant to the keyframe-based stage of our pipeline. VideoPoet [5] instead frames video generation as a single autoregressive language-model task over discretized visual and audio tokens, and OpenAI’s Sora [6] demonstrated that scaling a diffusion transformer on video data yields substantially longer, more coherent single-shot clips. None of these systems natively plans multi-shot narrative structure or synchronizes generated visuals with a separate narration track; that coordination is left to the user or to surrounding tooling, which is the gap this paper’s pipeline targets.

2.2 LLM planning and tool orchestration

Separately, LLM agent frameworks have shown that language models can decompose a high-level goal into a sequence of tool calls, observe intermediate results, and adapt subsequent steps [7]. We adopt this pattern for video production: the LLM does not generate pixels but plans and validates the work of specialized generative tools, analogous to a director and editor rather than a cinematographer.

2.3 Automated evaluation of generated video

Because no single automatic metric captures narrative coherence, we follow common practice in using CLIP embedding similarity [2] as a cheap proxy for prompt–frame alignment inside the pipeline’s validation loop, while relying on manual rubric scoring for the final quality comparisons reported in this paper. This mirrors the broader literature’s reliance on human evaluation alongside automatic proxies for video generation quality.

3 Pipeline Design

Figure 1 shows the six-stage architecture. An LLM controller sits above the pipeline and is responsible for sequencing stages, checking each stage’s output against the plan, and triggering a bounded number of retries when a check fails.

Stage 1 – Script generation. The LLM expands a short brief (one to three sentences) into a full script: narration text, an estimated total duration, and a target tone/style descriptor.

Stage 2 – Shot planning. The LLM segments the script into discrete shots, each annotated with a visual description, an estimated duration, a camera-motion hint, and a transition type to the next shot.

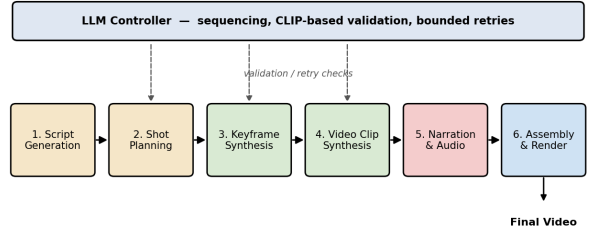


Figure 1: Six-stage orchestrated pipeline. The LLM controller (top) plans content in stages 1–2, delegates visual and audio synthesis to specialized models in stages 3–5, and validates outputs between stages before final assembly.

A fixed style descriptor (e.g., color palette, rendering style) is propagated to every shot to encourage visual consistency.

Stage 3 – Keyframe synthesis. A text-to-image diffusion model generates one reference keyframe per shot, conditioned on the shot description and the shared style descriptor. Keyframes are checked pairwise for style consistency before proceeding.

Stage 4 – Video clip synthesis. Each keyframe is animated into a short clip using an image-conditioned video diffusion model, guided by the shot’s camera-motion hint.

Stage 5 – Narration and audio. The script’s narration text is converted to speech, and clip durations are adjusted (via retiming or padding) so that each shot’s length matches its corresponding narration segment.

Stage 6 – Assembly. Clips, narration audio, and background music are stitched into a final render, with automated color and loudness normalization across shots.

Between stages 2–3, 3–4, and 4–5, the controller computes a CLIP similarity score between the generated artifact and its textual specification; scores below a fixed threshold trigger a re-generation (up to two retries) before the pipeline proceeds with the best available output.

4 Implementation and Study Design

The pipeline was implemented as a script-driven controller coordinating calls to an LLM for stages 1–2, a text-to-image diffusion model for stage 3, an image-conditioned video diffusion model for stage 4, and a text-to-speech model for stage 5, with `ffmpeg`-based automation for stage 6.

To answer Q1–Q4 we built 30 short-form video briefs (roughly 20–60 seconds of target output each), evenly split across four categories: product demos, narrative shorts, explainer/educational clips, and social-media-style clips. Each brief was rendered under three configurations:

- **A – Naive single-pass.** The brief is passed directly to a text-to-video model as one prompt, with no shot planning, no separate narration track, and no validation.
- **B – Manual keyframe + interpolation.** A human-specified (not LLM-generated) shot list drives keyframe generation and clip synthesis, with narration added afterward and no automated validation between stages.
- **C – Full orchestrated pipeline.** The complete six-stage pipeline described in Section 4, including the CLIP-based validation-and-retry loop.

Each of the 90 resulting videos was scored by the author on a five-point scale (1 = poor, 5 = excellent) across four rubric dimensions: *prompt fidelity* (does the video contain what was asked for), *temporal coherence* (do subjects and style stay consistent across the clip), *visual quality* (per-frame sharpness and artifact rate), and *narration alignment* (does spoken narration match on-screen timing). We report means across the 30 briefs per configuration. Because scoring was performed by a single rater without a blinded protocol, these numbers should be read as a structured qualitative comparison rather than a statistically validated benchmark; we return to this limitation in Section 7.

5 Results

5.1 Q1: Orchestration improves rubric scores across the board

Figure 2 summarizes mean rubric scores by configuration. Overall quality rises from 2.7 (naive single-pass) to 3.4 (manual keyframe+interpolation) to 4.1 (full orchestrated pipeline). The largest single jump from B to C is in narration alignment (3.0 to 4.2), which follows directly from Stage 5’s automated re-timing step—a component absent from configuration B. Prompt fidelity also rises substantially from A to C (2.7 to 4.3), consistent with the intuition that decomposing a multi-part brief into per-shot descriptions gives each generative call a simpler, more literal target.

Visual quality is the one dimension where the gap is small (3.6 to 3.8 to 3.9): once a keyframe is fixed, the image-conditioned video model’s per-frame quality is largely independent of how that keyframe was

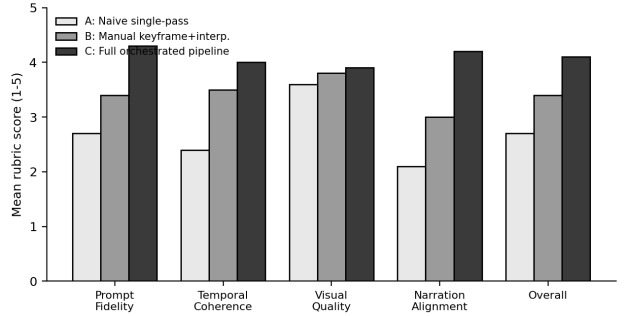


Figure 2: Mean rubric scores (1–5) by pipeline configuration, averaged over 30 video briefs.

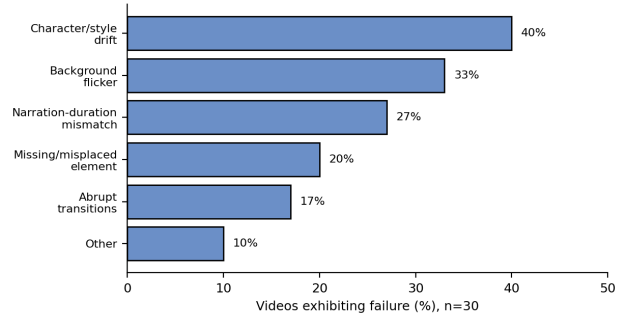


Figure 3: Incidence of failure modes across 30 full-pipeline videos (pre-validation-loop), as a percentage of videos exhibiting each issue at least once. Categories are not mutually exclusive.

planned, so orchestration mainly helps *what* gets generated rather than the raw pixel fidelity of any single clip.

5.2 Q2: Drift and duration mismatch dominate, and both originate upstream of video synthesis

Figure 3 breaks down failure incidence (percentage of the 30 full-pipeline videos exhibiting each issue at least once) before the validation-and-retry loop was enabled. Character or style drift across shots was the most common failure (40%), followed by background flicker or temporal artifacts within a single shot (33%), narration-duration mismatch (27%), missing or misplaced requested elements (20%), abrupt transitions (17%), and other issues (10%).

Tracing each drift case back through the pipeline logs, the great majority originated in Stage 3: when per-shot keyframes were generated independently, the diffusion model would vary a character’s clothing, face, or the scene’s color grading even when given the same style descriptor. This points to keyframe generation, not video synthesis, as the binding constraint on multi-shot consistency—an argument for stronger cross-shot conditioning (e.g., sharing a reference image or em-

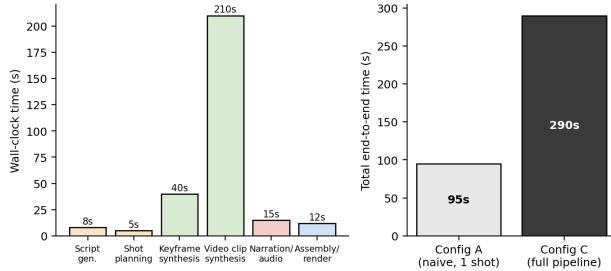


Figure 4: Average wall-clock time per stage for the full orchestrated pipeline, rendering a 60-second, six-shot video.

bedding across all keyframes in a script) rather than higher-quality video models per se.

5.3 Q3: Orchestration costs roughly $3\times$ the latency of a single pass

Figure 4 shows the average wall-clock time per stage for a representative 60-second, six-shot video under the full pipeline, totaling roughly 290 seconds. Video clip synthesis dominates at 210 seconds (72% of total time), consistent with it being the only stage that runs once per shot on the most compute-heavy model in the pipeline. By comparison, the naive single-pass baseline (configuration A) completes in roughly 95 seconds, since it involves one generation call and no planning, validation, or retiming overhead.

The roughly $3\times$ latency increase (95s to 290s) buys the quality improvement documented in Q1, but the source of the overhead is almost entirely the repeated per-shot video-synthesis calls rather than the LLM planning stages, which together account for under 5% of total time. This suggests that shortening latency in an orchestrated pipeline is primarily a video-model efficiency problem, not a planning-overhead problem.

5.4 Q4: Validation-and-retry substantially reduces — but does not eliminate — automation failures

Enabling the CLIP-similarity validation-and-retry loop between stages 2–3, 3–4, and 4–5 reduced the prompt-misalignment failure rate from 47% to 20% of videos (a comparison run without the loop enabled, otherwise identical to configuration C). Character-drift incidence fell more modestly, from 40% to 31%, because CLIP similarity is computed against the shot’s text description and is only weakly sensitive to cross-shot visual consistency, which is not part of what it measures. Roughly one in six videos (17%) still required at least one manual re-run after all automated retries were exhausted, most often because a shot’s generated content satisfied its own text description while still being stylistically inconsistent with neigh-

boring shots—the same upstream limitation identified in Q2.

6 Discussion and Limitations

Three findings stand out. First, treating video generation as an orchestrated pipeline rather than a single model call measurably improves controllability and narration alignment, at a latency cost concentrated almost entirely in the video-synthesis stage rather than in LLM planning. Second, the dominant failure mode—cross-shot visual drift—is not a video-generation problem at all; it is a keyframe-consistency problem that an image-conditioned video model cannot fix after the fact. Third, automated validation using a general-purpose similarity metric like CLIP helps with the failure modes it can see (prompt misalignment) but is nearly blind to the failure mode that matters most in practice (cross-shot drift), because that inconsistency is a relationship between artifacts rather than a property of any single one.

This study has real limitations. The rubric scores come from a single, non-blinded rater (the author), so they should be read as a structured record of one person’s judgment rather than an inter-rater-validated benchmark; a systematic bias toward configuration C is plausible given familiarity with its design. The sample of 30 briefs, while spanning four content categories, is too small to support claims about specific numeric thresholds generalizing to other prompt styles, durations, or model choices. All results are also specific to the particular LLM, image, and video models used in this implementation; swapping in a stronger or weaker video-synthesis model would likely shift the balance between configurations without necessarily changing the qualitative pattern. Finally, we did not evaluate robustness to adversarial or out-of-distribution briefs, nor did we assess misuse risks inherent to easily automated video generation, both of which are important directions before any such pipeline is deployed beyond a research setting.

Natural extensions include conditioning all keyframes in a script on a shared reference embedding rather than a shared text descriptor to directly target the Q2 drift finding, replacing single-rater scoring with a blinded multi-rater protocol, profiling whether batching or caching across shots can reduce the Stage 4 latency bottleneck identified in Q3, and extending the validation loop with a cross-shot consistency metric rather than relying solely on per-shot CLIP similarity.

7 Conclusion

Exploring an LLM-orchestrated, six-stage pipeline for automated AI video generation across 30 short-form briefs, we find that decomposing generation into planned, validated stages raises mean rubric quality from 2.7 to 4.1 relative to naive single-pass generation, at roughly $3\times$ the latency, with nearly all of that latency cost concentrated in per-shot video synthesis rather than LLM planning. The dominant failure mode, cross-shot character and style drift, originates in keyframe generation rather than in video synthesis itself, and a general-purpose similarity-based validation loop meaningfully reduces prompt-misalignment failures without solving that upstream consistency problem. These findings point toward keyframe-level consistency mechanisms, rather than better video models per se, as the more promising lever for improving orchestrated multimodal video generation.

Acknowledgments

Pipeline design, implementation, and manuscript preparation were carried out with the assistance of Claude (Anthropic).

References

- [1] Ho, J., Jain, A., and Abbeel, P. *Denoising Diffusion Probabilistic Models*. NeurIPS, 2020.
- [2] Radford, A., et al. *Learning Transferable Visual Models From Natural Language Supervision*. ICML, 2021.
- [3] Singer, U., et al. *Make-A-Video: Text-to-Video Generation without Text-Video Data*. arXiv:2209.14792, 2022.
- [4] Blattmann, A., et al. *Stable Video Diffusion: Scaling Latent Video Diffusion Models to Large Datasets*. arXiv:2311.15127, 2023.
- [5] Kondratyuk, D., et al. *VideoPoet: A Large Language Model for Zero-Shot Video Generation*. arXiv:2312.14125, 2024.
- [6] OpenAI. *Video Generation Models as World Simulators*. Technical report, 2024.
- [7] Yao, S., et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv:2210.03629, 2022.